

Unit 5

5.1 C Pointers:

- **Pointers** are one of the core components of the C programming language.
- A pointer can be used to store the **memory address** of other variables, functions, or even other pointers.
- The use of pointers allows low-level memory access, dynamic memory allocation, and many other functionality in C.

What is a Pointer in C?

- A pointer is defined as a derived data type that can store the address of other C variables or a memory location.
- We can access and manipulate the data stored in that memory location using pointers.
- As the pointers in C store the memory addresses, their size is independent of the type of data they are pointing to. This size of pointers in C only depends on the system architecture.

Syntax of C Pointers

The syntax of pointers is similar to the variable declaration in C, but we use the **(*) dereferencing operator** in the pointer declaration.

```
datatype * ptr;
```

where,

- **ptr** is the name of the pointer.
- **datatype** is the type of data it is pointing to.

The above syntax is used to define a pointer to a variable. We can also define pointers to functions, structures, etc.

How to Use Pointers?

The use of pointers in C can be divided into three steps:

1. **Pointer Declaration**
2. **Pointer Initialization**
3. **Pointer Dereferencing**

1. Pointer Declaration

In pointer declaration, we only declare the pointer but do not initialize it. To declare a pointer, we use the **(*) dereference operator** before its name.

Example:

```
int *ptr;
```

The pointer declared here will point to some random memory address as it is not initialized. Such pointers are called wild pointers.

2. Pointer Initialization

Pointer initialization is the process where we assign some initial value to the pointer variable. We generally use the **(&)** **addressof operator** to get the memory address of a variable and then store it in the pointer variable.

Example:

```
int var = 20;  
int * ptr;  
ptr = &var;
```

We can also declare and initialize the pointer in a single step. This method is called **pointer definition** as the pointer is declared and initialized at the same time.

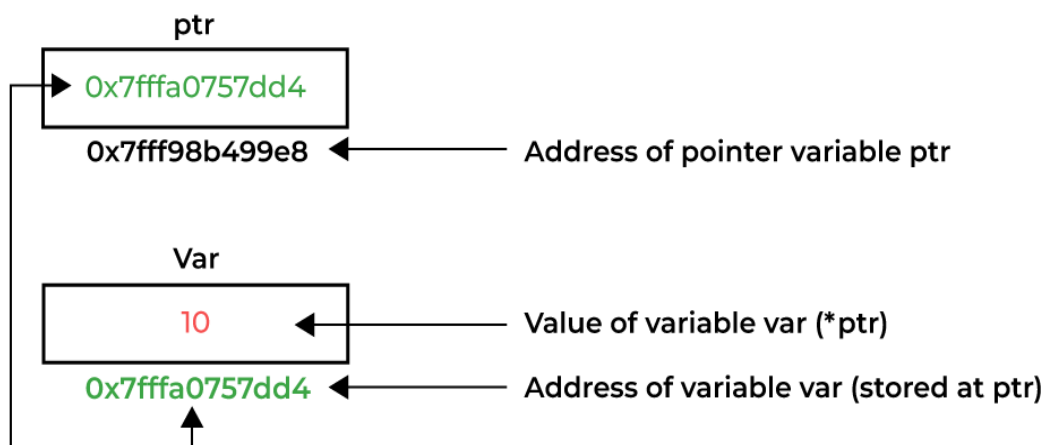
Example:

```
int *ptr = &var;
```

Note: It is recommended that the pointers should always be initialized to some value before starting using it. Otherwise, it may lead to number of errors.

3. Pointer Dereferencing

Dereferencing a pointer is the process of accessing the value stored in the memory address specified in the pointer. We use the same **(*)** **dereferencing operator** that we used in the pointer declaration.



C Pointer Example:

```
// C program to illustrate Pointers
```

```
#include <stdio.h>
```

```
void CWIT()
```

```

{
    int var = 10;

    // declare pointer variable

    int* ptr;

    // note that data type of ptr and var must be same

    ptr = &var;

    // assign the address of a variable to a pointer

    printf("Value at ptr = %p \n", ptr);

    printf("Value at var = %d \n", var);

    printf("Value at *ptr = %d \n", *ptr);
}

// Driver program

int main()
{
    CWIT();

    return 0;
}

```

Output:

Value at ptr = 0x7fff1038675c

Value at var = 10

Value at *ptr = 10

Types of Pointers in C

Pointers in C can be classified into many different types based on the parameter on which we are defining their types. If we consider the type of variable stored in the memory location pointed by the pointer, then the pointers can be classified into the following types:

1. Integer Pointers

As the name suggests, these are the pointers that point to the integer values.

Syntax:

```
int *ptr;
```

These pointers are pronounced as **Pointer to Integer**.

Similarly, a pointer can point to any primitive data type. It can point also point to derived data types such as arrays and user-defined data types such as structures.

2. Array Pointer

Pointers and Array are closely related to each other. Even the array name is the pointer to its first element. They are also known as Pointer to Arrays. We can create a pointer to an array using the given syntax.

Syntax:

```
char *ptr = &array_name;
```

3. Structure Pointer

The pointer pointing to the structure type is called Structure Pointer or Pointer to Structure. It can be declared in the same way as we declare the other primitive data types.

Syntax:

```
struct struct_name *ptr;
```

In C, structure pointers are used in data structures such as linked lists, trees, etc.

4. Function Pointers

Function pointers point to the functions. They are different from the rest of the pointers in the sense that instead of pointing to the data, they point to the code. Let's consider a function prototype – **int func (int, char)**, the function pointer for this function will be

Syntax:

```
int (*ptr)(int, char);
```

Note: The syntax of the function pointers changes according to the function prototype.

5. Double Pointers

In C language, we can define a pointer that stores the memory address of another pointer. Such pointers are called double-pointers or pointers-to-pointer. Instead of pointing to a data value, they point to another pointer.

Syntax:

```
datatype ** pointer_name;
```

Dereferencing Double Pointer

```
*pointer_name; // get the address stored in the inner level pointer
```

```
**pointer_name; // get the value pointed by inner level pointer
```

Note: In C, we can create multi-level pointers with any number of levels such as – *****ptr3, ****ptr4, *****ptr5** and so on.

6. NULL Pointer

The Null Pointers are those pointers that do not point to any memory location. They can be created by assigning a NULL value to the pointer. A pointer of any type can be assigned the NULL value.

Syntax:

```
data_type *pointer_name = NULL; or pointer_name = NULL
```

It is said to be good practice to assign NULL to the pointers currently not in use.

Advantages of Pointers

Following are the major advantages of pointers in C:

- Pointers are used for dynamic memory allocation and deallocation.
- An Array or a structure can be accessed efficiently with pointers
- Pointers are useful for accessing memory locations.
- Pointers are used to form complex data structures such as linked lists, graphs, trees, etc.
- Pointers reduce the length of the program and its execution time as well.

Disadvantages of Pointers

Pointers are vulnerable to errors and have following disadvantages:

- Memory corruption can occur if an incorrect value is provided to pointers.
- Pointers are a little bit complex to understand.
- Pointers are majorly responsible for memory leaks in C.
- Pointers are comparatively slower than variables in C.
- Uninitialized pointers might cause a segmentation fault.

5.2 Pointer Arithmetic in C

- Pointer Arithmetic is the set of valid arithmetic operations that can be performed on pointers.
- The pointer variables store the memory address of another variable. It doesn't store any value.
- Hence, there are only a few operations that are allowed to perform on Pointers in C language. The C pointer arithmetic operations are slightly different from the ones that we generally use for mathematical calculations.
- These operations are:
 1. Increment/Decrement of a Pointer
 2. Addition of integer to a pointer
 3. Subtraction of integer to a pointer
 4. Subtracting two pointers of the same type
 5. Comparison of pointers

1. Increment/Decrement of a Pointer

Increment: It is a condition that also comes under addition. When a pointer is incremented, it actually increments by the number equal to the size of the data type for which it is a pointer.

For Example:

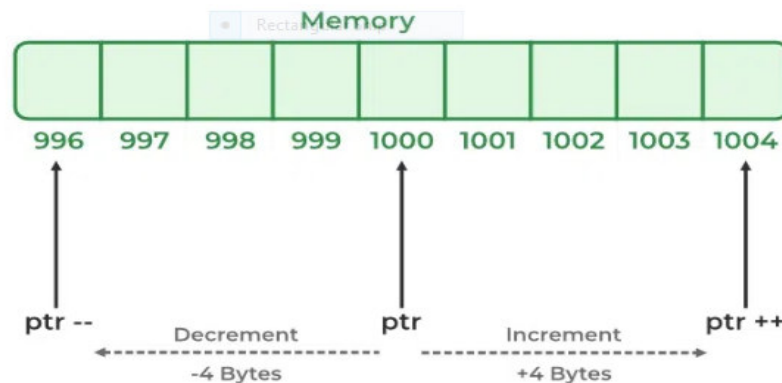
If an integer pointer that stores **address 1000** is incremented, then it will increment by 4(**size of an int**), and the new address will point to **1004**. While if a float type pointer is incremented then it will increment by 4(**size of a float**) and the new address will be **1004**.

Decrement: It is a condition that also comes under subtraction. When a pointer is decremented, it actually decrements by the number equal to the size of the data type for which it is a pointer.

For Example:

If an integer pointer that stores **address 1000** is decremented, then it will decrement by 4(**size of an int**), and the new address will point to **996**. While if a float type pointer is decremented then it will decrement by 4(**size of a float**) and the new address will be **996**.

Pointer Increment & Decrement



Note: It is assumed here that the architecture is 64-bit and all the data types are sized accordingly. For example, integer is of 4 bytes.

Example of Pointer Increment and Decrement

```
#include <stdio.h>
```

```
// pointer increment and decrement
```

```
//pointers are incremented and decremented by the size of the data type they point to
```

```
int main() {
```

```
    int a = 22;
```

```
    int *p = &a;
```

```
    printf("p = %u\n", p); // p = 6422288
```

```
    p++;
```

```
    printf("p++ = %u\n", p); //p++ = 6422292 +4 // 4 bytes
```

```
    p--;
```

```
    printf("p-- = %u\n", p); //p-- = 6422288    -4 // restored to original value
```

```

float b = 22.22;

float *q = &b;

printf("q = %u\n", q); //q = 6422284

q++;

printf("q++ = %u\n", q); //q++ = 6422288    +4 // 4 bytes

q--;

printf("q-- = %u\n", q); //q-- = 6422284    -4 // restored to original value


char c = 'a';

char *r = &c;

printf("r = %u\n", r); //r = 6422283

r++;

printf("r++ = %u\n", r); //r++ = 6422284    +1 // 1 byte

r--;

printf("r-- = %u\n", r); //r-- = 6422283    -1 // restored to original value

return 0;

}

```

Output:

```

p = 1441900792
p++ = 1441900796
p-- = 1441900792
q = 1441900796
q++ = 1441900800
q-- = 1441900796
r = 1441900791
r++ = 1441900792
r-- = 1441900791

```

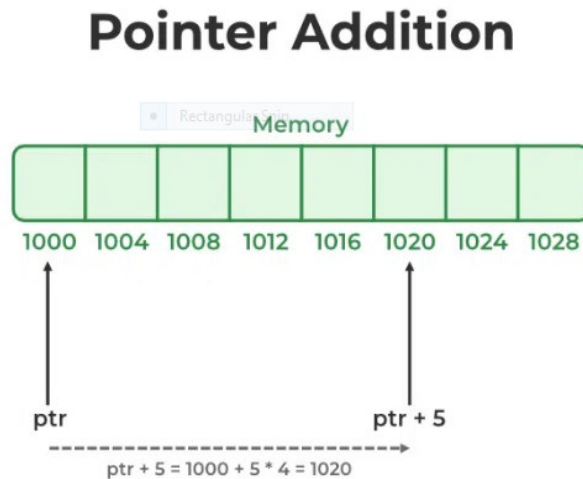
Note: Pointers can be outputted using %p, since, most of the computers store the address value in hexadecimal form using %p gives the value in that form. But for simplicity and understanding we can also use %u to get the value in Unsigned int form.

2. Addition of Integer to Pointer

When a pointer is added with an integer value, the value is first multiplied by the size of the data type and then added to the pointer.

For Example:

Consider the same example as above where the **ptr** is an **integer pointer** that stores **1000** as an address. If we add integer 5 to it using the expression, **ptr = ptr + 5**, then, the final address stored in the ptr will be **ptr = 1000 + sizeof(int) * 5 = 1020**.



Example of Addition of Integer to Pointer

// C program to illustrate pointer Addition

```
#include <stdio.h>
```

```
// Driver Code
```

```
int main()
```

```
{
```

```
    // Integer variable
```

```
    int N = 4;
```

```
    // Pointer to an integer
```

```
    int *ptr1, *ptr2;
```

```
    // Pointer stores the address of N
```

```
    ptr1 = &N;
```

```
    ptr2 = &N;
```


$$\}$$

```
// C program to illustrate pointer Subtraction
#include <stdio.h>
// Driver Code
```

```

int main()
{
    // Integer variable
    int N = 4;
    // Pointer to an integer
    int *ptr1, *ptr2;
    // Pointer stores the address of N
    ptr1 = &N;
    ptr2 = &N;
    printf("Pointer ptr2 before Subtraction: ");
    printf("%p \n", ptr2);
    // Subtraction of 3 to ptr2
    ptr2 = ptr2 - 3;
    printf("Pointer ptr2 after Subtraction: ");
    printf("%p \n", ptr2);
    return 0;
}

```

Output:

Pointer ptr2 before Subtraction: 0x7ffd718ffe5c

Pointer ptr2 after Subtraction: 0x7ffd718ffeb0

4. Subtraction of Two Pointers

The subtraction of two pointers is possible only when they have the same data type. The result is generated by calculating the difference between the addresses of the two pointers and calculating how many bits of data it is according to the pointer data type. The subtraction of two pointers gives the increments between the two pointers.

For Example:

Two integer pointers say **ptr1(address:1000)** and **ptr2(address:1004)** are subtracted. The difference between addresses is 4 bytes. Since the size of int is 4 bytes, therefore the **increment between ptr1 and ptr2** is given by **(4/4) = 1**.

Example of Subtraction of Two Pointer

```
// C program to illustrate Subtraction
```

```
// of two pointers
```

```
#include <stdio.h>
```

```
int main()
```

```

{
    int x = 6; // Integer variable declaration

    int N = 4;

    // Pointer declaration

    int *ptr1, *ptr2;

    ptr1 = &N; // stores address of N

    ptr2 = &x; // stores address of x

    printf(" ptr1 = %u, ptr2 = %u\n", ptr1, ptr2);

    // %p gives an hexa-decimal value,

    // We convert it into an unsigned int value by using %u

    // Subtraction of ptr2 and ptr1

    x = ptr1 - ptr2;

    // Print x to get the Increment

    // between ptr1 and ptr2

    printf("Subtraction of ptr1 & ptr2 is %d\n",x);

    return 0;
}

```

Output: Subtraction of ptr1 & ptr2 is 1

ptr1 = 2715594428, ptr2 = 2715594424

5. Comparison of Pointers

We can compare the two pointers by using the comparison operators in C. We can implement this by using all operators in C >, >=, <, <=, ==, !=. It returns true for the valid condition and returns false for the unsatisfied condition.

1. **Step 1:** Initialize the integer values and point these integer values to the pointer.
2. **Step 2:** Now, check the condition by using comparison or relational operators on pointer variables.
3. **Step 3:** Display the output.

Example of Pointer Comparison

```

// C Program to illustrates pointer comparison
#include <stdio.h>

```

```

int main()
{
// declaring array
int arr[5];
// declaring pointer to array name
int* ptr1 = &arr;
// declaring pointer to first element
int* ptr2 = &arr[0];

if (ptr1 == ptr2) {
    printf("Pointer to Array Name and First Element are Equal.");
}
else {
    printf("Pointer to Array Name and First Element are not Equal.");
}
return 0;
}

```

Output:

Pointer to Array Name and First Element are Equal.

Pointer	Array
A pointer is a derived data type that can store the address of other variables.	An array is a homogeneous collection of items of any type such as int, char, etc.
Pointers are allocated at run time.	Arrays are allocated at runtime.
The pointer is a single variable.	An array is a collection of variables of the same type.
Dynamic in Nature	Static in Nature.

5.3 Pointer to an Array / Array Pointer

Pointers and Array are closely related to each other. Even the array name is the pointer to its first element. They are also known as Pointer to Arrays. We can create a pointer to an array using the given syntax.

Syntax:

```
char *ptr = &array_name;
```

```
#include<stdio.h>
```

```
int main()
```

```
{
```

```
int arr[5] = { 1, 2, 3, 4, 5 };
```

```
int *ptr = arr;
```

```
printf("%p\n", ptr);
```

```
return 0;
```

```
}
```

In the above program, we have a pointer *ptr* that points to the 0th element of the array. Similarly, we can also declare a pointer that can point to whole array instead of only one element of the array. This pointer is useful when talking about multidimensional arrays.

Syntax: data_type (*var_name)[size_of_array];

Here:

- **data_type** is the type of data that the array holds.
- **var_name** is the name of the pointer variable.
- **size_of_array** is the size of the array to which the pointer will point.

Example:

```
int (*ptr)[10];
```

Note: The pointer that points to the 0th element of array and the pointer that points to the whole array are totally different. The following program shows this:

// C program to understand difference between pointer to an integer and pointer to an array of integers.

```
#include<stdio.h>
```

```
int main()
```

```
{
```

```
    // Pointer to an integer
```

```
    int *p;
```

```
    // Pointer to an array of 5 integers
```

```
    int (*ptr)[5];
```

```
    int arr[5];
```

```
    // Points to 0th element of the arr.
```

```

p = arr;

// Points to the whole array arr.
ptr = &arr;

printf("p = %p, ptr = %p\n", p, ptr);

p++;
ptr++;

printf("p = %p, ptr = %p\n", p, ptr);

return 0;
}

```

Output:

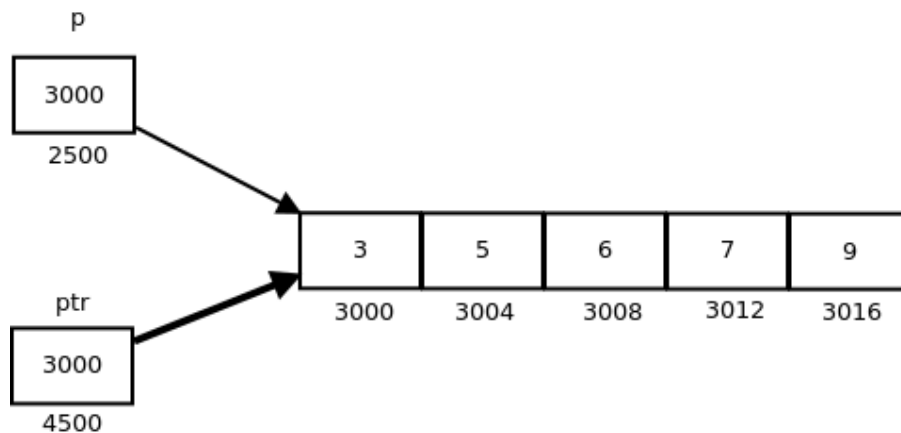
p = 0x7fff6463e890, ptr = 0x7fff6463e890

p = 0x7fff6463e894, ptr = 0x7fff6463e8a4

Here, **p** is pointer to 0th element of the array *arr*, while **ptr** is a pointer that points to the whole array *arr*.

- The base type of *p* is int while base type of *ptr* is 'an array of 5 integers'.
- We know that the pointer arithmetic is performed relative to the base size, so if we write *ptr++*, then the pointer *ptr* will be shifted forward by 20 bytes.

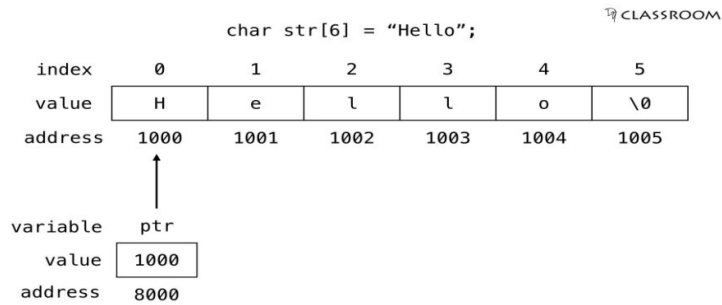
The following figure shows the pointer *p* and *ptr*. The darker arrow denotes a pointer to an array.



5.4 Pointer and text string

A pointer to a string in C can be used to point to the base address of the string array, and its value can be dereferenced to get the value of the string. To get the value of the string array is iterated using a while loop until a null character is encountered.

Syntax: char *ptr = str;



The pointer variable ptr is allocated memory address 8000 and it holds the address of the string variable str i.e., 1000.

Accessing string via pointer

```
#include <stdio.h>

int main(void) {

    // string variable
    char str[6] = "Hello";

    // pointer variable
    char *ptr = str;

    // print the string
    while(*ptr != '\0') {
        printf("%c", *ptr);

        // move the ptr pointer to the next memory location
        ptr++;
    }

    return 0;
}
```

Output:

Hello

Using pointer to store string

We can achieve the same result by creating a character pointer that points at a string value stored at some memory location.

```
#include <stdio.h>

int main(void) {

    // pointer variable to store string
    char *strPtr = "Hello";

    // temporary pointer variable
    char *t = strPtr;
```

```

// print the string
while(*t != '\0') {
    printf("%c", *t);

    // move the t pointer to the next memory location
    t++;
}

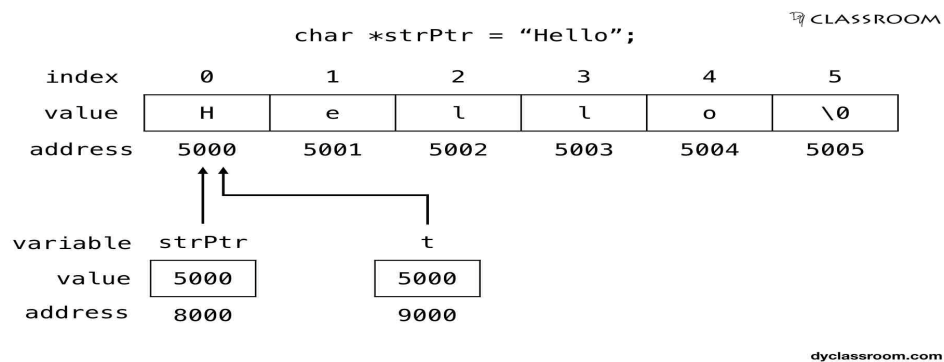
return 0;
}

```

Output:

Hello

Note: In the above code we are using another character pointer `t` to print the characters of the string as because we don't want to lose the starting address of the string "Hello" which is saved in pointer variable `strPtr`.



- In the above image the string "Hello" is saved in the memory location 5000 to 5005.
- The pointer variable `strPtr` is at memory location 8000 and is pointing at the string address 5000.
- The temporary variable is also assigned the address of the string so, it too holds the value 5000 and points at the starting memory location of the string "Hello".

5.5 Function handling using pointers/ Function pointer:

Function pointers point to the functions. They are different from the rest of the pointers in the sense that instead of pointing to the data, they point to the code. Let's consider a function prototype – **int func (int, char)**, the function pointer for this function will be

Syntax:

```
int (*ptr)(int, char);
```

Note: The syntax of the function pointers changes according to the function prototype.

In C, like normal data pointers (int *, char *, etc), we can have pointers to functions. Following is a simple example that shows declaration and function call using function pointer.

```
#include <stdio.h>

// A normal function with an int parameter

// and void return type

void fun(int a)

{

    printf("Value of a is %d\n", a);

}

int main()

{

    // fun_ptr is a pointer to function fun()

    void (*fun_ptr)(int) = &fun;

    /* The above line is equivalent of following two

    void (*fun_ptr)(int);

    fun_ptr = &fun;

    */

    // Invoking fun() using fun_ptr

    (*fun_ptr)(10);

    return 0;

}
```

Output:

Value of a is 10

Following are some interesting facts about function pointers.

1. Unlike normal pointers, a function pointer points to code, not data. Typically a function pointer stores the start of executable code.
2. Unlike normal pointers, we do not allocate de-allocate memory using function pointers.
3. A function's name can also be used to get functions' address. For example, in the below program, we have removed address operator '&' in assignment. We have also changed function call by removing *, the program still works.

```

#include <stdio.h>
// A normal function with an int parameter
// and void return type
void fun(int a)
{
    printf("Value of a is %d\n", a);
}

int main()
{
    void (*fun_ptr)(int) = fun; // & removed

    fun_ptr(10); // * removed

    return 0;
}

```

Output:

Value of a is 10

5.6 Structure Pointer in C

A structure pointer is defined as the pointer which points to the address of the memory block that stores a structure known as the structure pointer. Complex data structures like Linked lists, trees, graphs, etc. are created with the help of structure pointers. The structure pointer tells the address of a structure in memory by pointing the variable to the structure variable.

// C program to demonstrate structure pointer

```

#include <stdio.h>

struct point {
    int value;
};

int main()
{
    struct point s;

    // Initialization of the structure pointer

    struct point* ptr = &s;

```

```
        return 0;
    }
}
```

In the above code **s** is an instance of struct point and **ptr** is the struct pointer because it is storing the address of struct point.

Accessing the Structure Member with the Help of Pointers

There are two ways to access the members of the structure with the help of a structure pointer:

1. With the help of (*) asterisk or indirection operator and (.) dot operator.
2. With the help of (->) Arrow operator.

Below is the program to access the structure members using the structure pointer with the help of the dot operator.

// C Program to demonstrate Structure pointer

```
#include <stdio.h>

#include <string.h>

struct Student {
    int roll_no;
    char name[30];
    char branch[40];
    int batch;
};

int main()
{
    struct Student s1;
    struct Student* ptr = &s1;
    s1.roll_no = 27;
    strcpy(s1.name, "Kamlesh Joshi");
    strcpy(s1.branch, "Computer Science And Engineering");
    s1.batch = 2019;
    printf("Roll Number: %d\n", (*ptr).roll_no);
    printf("Name: %s\n", (*ptr).name);
}
```

```

        printf("Branch: %s\n", (*ptr).branch);

        printf("Batch: %d", (*ptr).batch);

        return 0;

}

```

Below is the program to access the structure members using the structure pointer with the help of the Arrow operator. In this program, we have created a Structure Student containing structure variable s. The Structure Student has roll_no, name, branch, and batch.

// C Program to demonstrate Structure pointer

```
#include <stdio.h>
```

```
#include <string.h>
```

// Creating Structure Student

```
struct Student {
```

```
    int roll_no;
```

```
    char name[30];
```

```
    char branch[40];
```

```
    int batch;
```

```
};
```

// variable of structure with pointer defined

```
struct Student s, *ptr;
```

```
int main()
```

```
{
```

```
    ptr = &s;
```

// Taking inputs

```
    printf("Enter the Roll Number of Student\n");
```

```
    scanf("%d", &ptr->roll_no);
```

```
    printf("Enter Name of Student\n");
```

```
    scanf("%s", &ptr->name);
```

```
    printf("Enter Branch of Student\n");
```

```
    scanf("%s", &ptr->branch);
```

```
printf("Enter batch of Student\n");  
  
scanf("%d", &ptr->batch);  
  
// Displaying details of the student  
  
printf("\nStudent details are: \n");  
  
printf("Roll No: %d\n", ptr->roll_no);  
  
printf("Name: %s\n", ptr->name);  
  
printf("Branch: %s\n", ptr->branch);  
  
printf("Batch: %d\n", ptr->batch);  
  
return 0;  
  
}
```

Output:

Enter the Roll Number of Student

27

Enter Name of Student

Kamlesh_Joshi

Enter Branch of Student

Computer_Science_And_Engineering

Enter batch of Student

2019

Student details are:

Roll No: 27

Name: Kamlesh_Joshi

Branch: Computer_Science_And_Engineering

Batch: 2019